UNIVERSIDAD NACIONAL DE SAN ANTONIO ABAD DEL CUSCO

DEPARTAMENTO ACADÉMICO DE INGENIERÍA INFORMÁTICA

REDES DE COMPUTADORAS 2

Lab 02: Programación de Sockets en UDP con Python

Leunam

"Dime y lo olvidaré. Muéstrame y lo recordaré. Involúcrame y lo entenderé"

Proverbio chino

---

1. OBJETIVOS:

1.1. Entender lo básico de la comunicación por redes.
1.2. Programar aplicaciones de red usando sockets.
1.3. Aprender cómo construir aplicaciones cliente/servidor que se comunican usando sockets
1.4. Aprender el uso de sockets en Python.

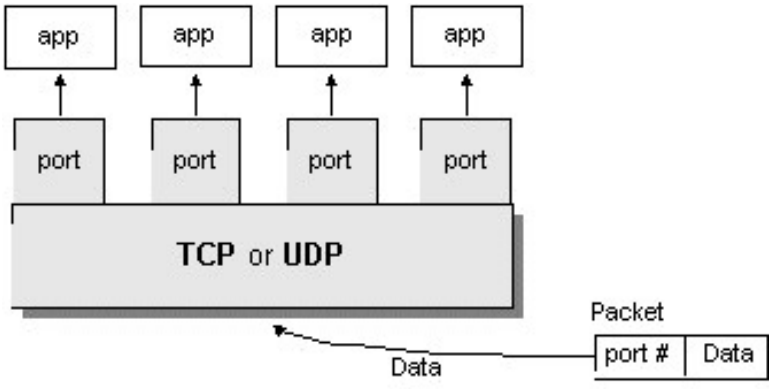
2. BASE TEORICA COMPLENTARIA:

UDP: User Datagram Protocol (Protocolo de Datagrama del Usuario):

- El protocolo UDP estipula comunicación que no está garantizada entre 2 aplicaciones en la red.
  - UDP no está basada en la conexión como TCP.
  - Más bien, envía paquetes de datos independientes, llamados datagramas, desde una aplicación a otra.
  - Enviar datagramas es muy parecido a enviar una carta a través del servicio postal: el orden de entrega no es importante y no está garantizado, y cada mensaje es independiente de cualquier otro.

- Ejemplo 1: un servidor de reloj que envía la hora actual a su cliente cuando le es solicitado hacer así.
  - Si el cliente pierde un paquete, realmente no tiene sentido reenviarlo porque la hora será incorrecta cuando el cliente lo reciba en el segundo intento.
  - Si el cliente hace 2 peticiones y recibe paquetes desde el servidor fuera de orden, realmente no importa porque el cliente puede entender que los paquetes están fuera de orden y hacer otra petición.
- Ejemplo 2: comando ping, servicio que no necesita la garantía de un canal confiable.
  - El propósito del comando ping es testear la comunicación entre 2 programas sobre la red.
  - De hecho, ping necesita saber acerca de los paquetes "caídos" o fuera de orden para determinar cuan buena o mala es la conexión.

- En una comunicación basado-en-datagrama tal como UDP, el paquete datagrama contiene el número de puerto de su destino y UDP encamina el paquete a la aplicación apropiada.



- Algunas aplicaciones que se escriben para comunicarse sobre la red no requerirán el canal confiable, punto-a-punto proveído por TCP.
    - Más bien, estas aplicaciones podrían beneficiarse de un modo de comunicación que entrega paquetes de información independientes cuyo arribo y orden de arribo no están garantizados.
- El protocolo UDP provee un modo de comunicación de red por medio del cual las aplicaciones envían paquetes de datos, llamados datagramas uno al otro.
- Las aplicaciones q' se comunican vía datagramas envían y reciben paquetes de información completamente independientes.
    - Estos clientes y servidores no tienen y no necesitan un canal punto-a-punto dedicado.
    - La entrega de datagramas a sus destinos no está garantizada, tampoco está el orden de su arribo.

Datagrama:
- Un datagrama es un mensaje independiente, autocontenido enviado sobre la red cuyo arribo, hora de llegada, y contenido no están garantizados.

- UDP: no orientado a "conexión" entre cliente y servidor.
    - No Handshaking.
    - El emisor explícitamente liga dirección IP y puerto de destino a cada paquete.
    - El servidor debe de extraer dirección IP, puerto del emisor desde el paquete recibido.
- UDP: datos transmitidos puede ser recibidos fuera de orden, o ser perdidos.
- UDP provee transferencia **no confiable** de grupos de bytes ("datagramas") entre el cliente y el servidor

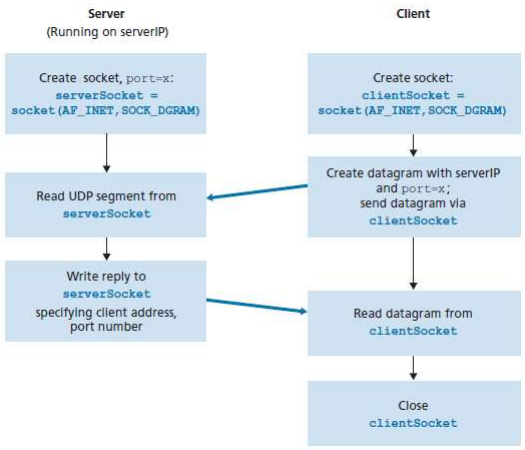
3. RECURSOS:

3.1. Python.
3.2. Ubuntu/Windows.

4. DESARROLLO DE LA PRACTICA:

Recuerde que los procesos corriendo en diferentes máquinas se comunican el uno con el otro mediante el envío de mensajes dentro de los sockets.
En el presente trabajo de laboratorio se implementará en UDP tanto un cliente que envía datos en minúscula a un servidor, como un servidor que recibe los datos en minúscula desde un cliente y lo devuelve de regreso, pero en mayúsculas. El lenguaje a usar será Python.

4.1. Escribir una aplicación cliente/servidor que implemente el servicio “CAMBIAR DATOS QUE ESTAN EN MINUSCULAS A MAYUSCULAS”.
Aplicación Cliente-Servidor usando UDP:



APLICACIÓN CLIENTE:

ClienteUDP.py

```
# ClienteUDP.py

# incluir librería de socket de Python
from socket import *

# inicializar variables
idServidor = 'id_Servidor'              # reemplazar con la dirección IP del servidor
puertoServidor = 12000

# crear socket cliente UDP
#   1er. parámetro: familia de direcciones: AF_INET indica IPv4
#   2do. parámetro: tipo de socket: SOCK_DGRAM significa socket UDP
socketCliente = socket(AF_INET, SOCK_DGRAM)

# obtener alguna entrada desde el teclado del usuario
mensaje = input('Entrar una sentencia en minúsculas: ')

# adjuntar nombre del servidor y el puerto al mensaje
#   y enviarlo hacia el socket
socketCliente.sendto(mensaje.encode(), (idServidor, puertoServidor))

# leer caracteres replicados que llegaron al socket hacia un string
#   (los datos son puestos en la variable mensajeModificado, y la dirección
#   fuente del paquete en la variable direccionServidor)
mensajeModificado, direccionServidor = socketCliente.recvfrom(2048)

# imprimir string recibido
print ('Socket del servidor = ', direccionServidor)
print(mensajeModificado.decode())

# cerrar socket
socketCliente.close()
```

APLICACIÓN SERVIDOR:

ServidorUDP.py

```
# ServidorUDP.py

from socket import *

# inicializar variables
puertoServidor = 12000

# crear socket servidor UDP
socketServidor = socket(AF_INET, SOCK_DGRAM)

# enlazar socket al nro. de puerto local 12000
socketServidor.bind(('', puertoServidor))

# mensaje: que el servidor está listo
print('El servidor está listo para recibir')

# lazo 'eterno'
while True:
        # leer desde el socket UDP para cargar el mensaje,
```

```
#   y obtener la dirección del cliente (el IP y el nro. puerto del cliente)
mensaje, direccionCliente = socketServidor.recvfrom(2048)

# cambiar mensaje a mayúsculas
mensajeModificado = mensaje.decode().upper()

# enviar mensaje modificado de regreso al cliente
socketServidor.sendto(mensajeModificado.encode(), direccionCliente)
```

- En Windows/Ubuntu abrir 2 ventanas de CLI (Interface de Línea de Comandos) o de Terminal, y en cada ventana cambiarse al directorio donde están los archivos Python.

  ServidorUDP.py
  ClienteUDP.py

- En Windows/Ubuntu, en la ventana del servidor ejecutar el programa Servidor con tipear lo siguiente:

  Windows:
  python ServidorUDP.py
  o
  py ServidorUDP.py

  Ubuntu:
  python3 ServidorUDP.py

  (tipear un texto en minúsculas y presionar Enter)

- En Windows/Ubuntu, en la ventana del cliente ejecutar el programa Cliente con tipear lo siguiente:
  Windows:
  python ClienteUDP.py
  o
  py ClienteUDP.py

  Ubuntu:
  python3 ClienteUDP.py

  (tipear un texto en minúsculas y presionar Enter)

5. EJERCICIOS PROPUESTOS:

5.1. **Utilizar netcat (nc) para probar el servidor UDP:**

Objetivo: Verificar el funcionamiento del servidor UDP implementado en Python sin necesidad de ejecutar el cliente Python, usando una herramienta estándar de red.

Procedimiento:

a) Ejecutar el servidor UDP en una terminal:
   python3 ServidorUDP.py

b) En otra terminal, usar `netcat` para enviar un datagrama UDP al servidor. Por ejemplo:
   En Windows (si se tiene `netcat` instalado):

```
echo hola servidor | nc -u -w1 192.168.1.36 12000
```

En Ubuntu:

```
echo "hola servidor" | nc -u -w1 192.168.1.36 12000
```

c) Observar la respuesta del servidor.
`netcat` debería mostrar el texto en mayúsculas:

HOLA SERVIDOR

5.2. En Windows abra una ventana CLI (Interface de Línea de Comandos) e investigue sobre el comando `tracert`. Luego trate de encontrar un Servidor cualesquiera para el cual la traza que devuelve `tracert` contenga una cantidad de saltos mayor al máximo número de saltos predeterminado:
Nota: El estudiante que encuentre el mayor número de saltos tendrá la mejor ponderación para este ejercicio.
Los casos de saltos reciclados (saltos por nodos ya visitados) no serán válidos.

Tipo de trabajo	Individual
Tipo de informe	Digital: PDF
Fecha de entrega	dd/mm/aaaa